

Movie Recommendation System

Alexander Park, Shariq Rifai, Haozhe Shi

1. Motivation & Problem Statement

With so many movies out there and the length of a movie being such a large time commitment, it is difficult to find a new movie that you might like. It can also be a struggle to find a movie that has the same ‘vibe’ as another movie, and a viewer might not know what draws them to that specific ‘vibe.’ Streaming services already recommend shows and movies based on user preferences, such as the “Recommended” section on Netflix, but people remain unsatisfied with those recommendations and stay unsure of what to watch next. A good movie recommendation model with customizable inputs can solve this problem.

2. Approach

Our system takes a list of movies the user has previously enjoyed and an optional genre constraint, then recommends new movies with a yes/no feedback loop. If the user rejects a recommendation, the model learns from that feedback and returns a different candidate. The recommendation pipeline has two stages. **Stage 1 – Heuristic Scoring** compares candidate movies against the user’s liked-movie profile using four weighted features: shared actors (+1 each), shared genres (+2 each), IMDb rating (+1 per star), and shared plot keywords (+1 each). Movie metadata (actors, genres, rating, runtime) is fetched from the Open Movie Database (OMDB) API, and plot keywords are generated by prompting an LLM (Groq’s Llama-3.1-8B) to produce 10 descriptive keywords per plot. This stage yields an interpretable score breakdown that is shown to the user.

Stage 2 – Neural Network Refinement takes a 37-dimensional feature vector (normalized heuristic sub-scores, raw rating, one-hot runtime bins, and a 28-dim genre one-hot) and passes it through a 2-hidden-layer network (37→64→32→1) with ReLU activations, dropout (0.3), and a sigmoid output representing P(recommend). Implemented in PyTorch. After each user yes/no response, the network performs 10 Adam gradient steps (BCE loss, lr=0.01) on that single sample, enabling online learning. The model is pre-trained on 180 synthetic samples derived from 6 simulated taste profiles to avoid cold-start.

Assumptions & limitations: the candidate pool is drawn from OMDB keyword search (not a full database), OMDB free-tier is capped at 1000 requests/day, and LLM keyword quality depends on plot detail.

3. Key Results

Pre-training. We generated 180 synthetic (feature-vector, label) pairs from 30 hardcoded movies across 6 simulated taste profiles (action/sci-fi, drama/romance, comedy, horror/thriller, animation, crime/classic). A candidate was labeled positive if its heuristic total met or exceeded the median for that profile. Training the NN for 200 epochs with Adam (lr=0.01, BCE loss) converged from an initial loss of ~0.69 to 0.09 and achieved **95.0% training accuracy** (171/180 correct).

End-to-end qualitative example. Using real OMDB + LLM data with *Inception* and *The Dark Knight* as liked movies: *Interstellar* received a heuristic total of 14 (+6 genre, +8 rating) and NN confidence of **100%**, while *Forrest Gump* received a heuristic total of 10 (+2 genre, +8 rating) and NN confidence of only **9.0%**. Although the heuristic totals differ by just 4 points, the NN sharply distinguishes between the two based on the *combination* of features (specifically the genre-overlap channel), demonstrating that the network has learned to weight genre compatibility more strongly than raw rating.

Online learning response. When the simulated user rejected *Forrest Gump*, the NN’s score for that movie dropped from 9.0% to 0.04% after 10 gradient steps – a >200× decrease from a single negative feedback signal. Conversely, accepting *Interstellar* held its score at 100%, confirming that the saturation of strong matches is preserved under online updates. This behavior was consistent with our expectation that online learning should quickly penalize rejected candidates while reinforcing accepted ones.

Interpretability. Every recommendation is presented with its four-component heuristic breakdown (actors, genres, rating, keywords) alongside the NN probability, so users can see *why* a movie was suggested – a feature that distinguishes our system from opaque streaming-service recommenders.



4. Challenges & Lessons Learned

We definitely faced some challenges with communication since we all have our own schedules and were always busy with other work. There were some meetings we held in which one of us was unable to meet and had to be filled in afterwards. We addressed this by staying active in the group chat to communicate when we do/don't have time at certain points, and staying organized with a design document and repository. With this, even if we weren't all able to meet, we could catch up by reading updates on these platforms and keeping an open line of communication if any questions arise.

5. Individual Contributions

Member	Primary Contributions	Approx. %
Alex	System and environment design, project management, dataset	33%
Shariq	LLM setup, CLI user interface	33%
Haozhe	API setup, Neural network model implementation	33%

6. Repository & References

GitHub Repository: <https://github.com/alexpark0/CS4100-Group-Project>

Project Tracker: <https://github.com/users/alexpark0/projects/1/views/1>

Key References:

- [1] B. Fritz, “The Open Movie Database (OMDb) API,” omdbapi.com, 2024. RESTful web service used to fetch movie metadata (actors, genres, IMDb rating, runtime, plot).
- [2] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” NeurIPS, 2019. Deep-learning framework used to implement and train the 2-hidden-layer recommendation network.
- [3] Groq Inc., “Groq LPU Inference Engine and Llama-3.1 models,” groq.com, 2024. Low-latency LLM inference used to generate 10 plot keywords per movie via an OpenAI-compatible API.
- [4] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, 3rd ed., Springer, 2022. Foundational reference for content-based and hybrid recommendation approaches combining heuristic features with learned scoring.